

INF-452 Geometría Computacional — Guía de Estudio Certamen 1

Certamen: miércoles 22/04/2026, 11:05 hrs, Sala LPA **Contenidos:** Convex Hull 2D · Line Segment Intersection · Polygon Triangulation · Linear Programming · Orthogonal Range Queries **Modalidad:** Material abierto, sin calculadora. Mostrar todo el desarrollo.

Herramientas Matemáticas Transversales

Antes de entrar a cada unidad, hay primitivas geométricas que aparecen una y otra vez.

Orientación de tres puntos (Left Turn Test). Dados tres puntos $p = (px, py)$, $q = (qx, qy)$, $r = (rx, ry)$, calculamos el determinante:

$$\det = \begin{vmatrix} qx - px & rx - px \\ qy - py & ry - py \end{vmatrix} = (qx - px)(ry - py) - (rx - px)(qy - py)$$

- $\det > 0 \rightarrow r$ está a la **izquierda** de la línea dirigida $p \rightarrow q$ (left turn / counterclockwise)
- $\det < 0 \rightarrow r$ está a la **derecha** (right turn / clockwise)
- $\det = 0 \rightarrow$ los tres puntos son colineales

Esta operación cuesta $O(1)$ y es la base de casi todos los algoritmos del curso.

Intersección de dos segmentos. Dos segmentos $s_1 = p_1q_1$ y $s_2 = p_2q_2$ se intersectan si y solo si:

1. Los endpoints de s_2 están en lados opuestos de la línea que contiene s_1 , y viceversa.
2. O algún endpoint es colineal y cae dentro del otro segmento.

Esto se verifica con 4 tests de orientación, $O(1)$.

Orden lexicográfico. Para puntos en $\mathbb{R}^2$, se dice que $p < q$ si $px < qx$, o si $px = qx$ y $py < qy$. Se usa extensivamente para definir "izquierda-derecha" en sweep lines y en la ordenación de puntos para Convex Hull.

Unidad 1: Convex Hull 2D

1.1 Definiciones fundamentales

Conjunto convexo: Un subconjunto S del plano es convexo si para cualquier par de puntos $p, q \in S$, el segmento pq está completamente contenido en S .

Convex Hull (envolvente convexa): Dado un conjunto $S \subset \mathbb{R}^2$, $CH(S)$ es la intersección de todos los conjuntos convexos que contienen a S . Equivalentemente, es la intersección de todos los semiplanos cerrados que contienen a S .

Interpretación física: imagina un elástico estirado alrededor de clavos (los puntos). Al soltarlo, el elástico toma la forma de $CH(S)$.

Propiedad clave para la motivación: Dado un conjunto S de sustancias con fracciones de componentes A y B , se puede mezclar una sustancia q a partir de S si y solo si $q \in CH(S)$.

1.2 Output del problema

- **Input:** conjunto S de n puntos en $\mathbb{R}^2$.
- **Output:** lista de vértices de $CH(S)$ en orden (clockwise o counterclockwise).

Observación fundamental: Una arista dirigida (p, q) pertenece al $CH(S)$ si y solo si todos los demás puntos de S están estrictamente a la derecha de la línea dirigida $p \rightarrow q$ o sobre el segmento pq .

1.3 Algoritmo Ingenuo — FirstConvexHull: $O(n^3)$

```
FirstConvexHull(S):
    E ← ∅
    for each (p, q) ∈ S × S con p ≠ q:           // O(n²) pares
        valid ← true
        for each r ∈ S:                           // O(n) verificaciones
            if r NO está estrictamente a la derecha de p→q y r ∉ pq:
                valid ← false
        if valid:
            E ← E ∪ {(p, q)}
    Construir lista ordenada de vértices a partir de E.
```

Complejidad: $O(n^3)$. **Problema:** No es robusto. Falla con puntos colineales (3 puntos en una línea pueden crear ambigüedades).

1.4 Graham Scan (Upper/Lower Hull): $O(n \log n)$

La idea clave: dividir el cómputo en **upper hull** y **lower hull**.

```
UpperConvexHull(S):
    1.  $\langle p_1, p_2, \dots, p_n \rangle \leftarrow$  ordenar  $S$  lexicográficamente // O(n log n)
    2. L ←  $\langle p_1, p_2 \rangle$ 
    3. for i ← 3 to n:
        L.append(pi)
        while |L| > 2 AND los últimos 3 puntos en L hacen LEFT TURN:
            remover el penúltimo punto de L
    4. return L
```

Para el Lower Hull: se hace lo mismo pero iterando de p_n a p_1 (de derecha a izquierda), o equivalentemente iterando de izquierda a derecha y chequeando right turns.

Para CH completo: concatenar UpperHull + LowerHull (removiendo los puntos duplicados en los extremos).

Análisis de complejidad:

- Sorting: $O(n \log n)$
- Loop principal: cada punto se agrega a L exactamente una vez y se remueve a lo sumo una vez $\rightarrow$ el total de operaciones del while es $O(n)$ (argumento amortizado).
- **Total: $O(n \log n)$**

¿Por qué es robusto? Solo usa tests de orientación (left turn), no requiere computar intersecciones ni manejar casos degenerados complicados.

1.5 Algoritmos output-sensitive

- **Jarvis March (Gift Wrapping):** $O(n \cdot h)$ donde $h = |CH(S)|$ (número de vértices del hull). En cada paso, encuentra el punto que forma el menor ángulo. Bueno cuando h es pequeño.
- **Chan's Algorithm:** $O(n \log h)$. Combina Graham Scan en subgrupos con Jarvis March. Es óptimo.

1.6 Resumen de complejidades

Algoritmo	Tiempo	Nota
FirstConvexHull	$O(n^3)$	No robusto
Graham Scan	$O(n \log n)$	Robusto, óptimo en general
Jarvis March	$O(nh)$	Output-sensitive
Chan's	$O(n \log h)$	Óptimo output-sensitive

1.7 Cómo resolver ejercicios de Convex Hull

Tipo: "Ejecute Graham Scan sobre estos puntos"

1. Ordenar los puntos lexicográficamente.
2. Construir upper hull de izquierda a derecha, verificando left turns.
3. Construir lower hull de derecha a izquierda (o izquierda a derecha con right turns).
4. Concatenar.
5. Mostrar paso a paso qué puntos se agregan/remueven del stack.

Tipo: "Demuestre que X usa convex hull" Recordar que q se puede expresar como combinación convexa de $S \iff q \in CH(S)$.

Unidad 2: Line Segment Intersection

2.1 El Problema

Input: conjunto S de n segmentos cerrados no superpuestos en el plano. **Output:** todos los puntos donde al menos dos segmentos se intersectan, y para cada punto, reportar todos los segmentos que lo contienen.

Fuerza bruta: revisar todos los pares $\rightarrow O(n^2)$ intersecciones a revisar.

¿Se puede mejorar? Sí, con un **sweep line algorithm** (algoritmo de barrido).

2.2 Paradigma Sweep Line

Idea fundamental: una línea horizontal ℓ barre el plano de arriba hacia abajo. Solo nos importan los segmentos que son "vecinos" en la línea de barrido.

Observación clave: dos segmentos solo pueden intersectarse si en algún momento son adyacentes en el orden de la sweep line. Esto reduce dramáticamente las comparaciones.

Dos estructuras de datos:

1. **Event Queue Q** — cola de prioridad (balanced BST) que almacena los puntos evento ordenados de arriba a abajo (y de izquierda a derecha si misma y). Orden: $p < q \iff y_p > y_q$, o ($y_p = y_q$ y $x_p < x_q$).
2. **Status Structure T** — balanced BST que almacena los segmentos que cortan la sweep line, ordenados de izquierda a derecha.

2.3 Tipos de evento y conjuntos L , U , C

Para cada punto evento p se definen tres conjuntos:

- **$U(p)$:** segmentos cuyo **upper endpoint** (inicio) es $p \rightarrow$ segmentos que "comienzan" en p .
- **$L(p)$:** segmentos cuyo **lower endpoint** (fin) es $p \rightarrow$ segmentos que "terminan" en p .
- **$C(p)$:** segmentos cuyo interior contiene a $p \rightarrow$ segmentos que "cruzan" por p .

Criterio de intersección: p es un punto de intersección si $|L(p) \cup U(p) \cup C(p)| > 1$.

2.4 Algoritmo FindIntersections

```

FindIntersections(S):
  Q  $\leftarrow \emptyset$ ; T  $\leftarrow \langle \text{sentinelas } x = -\infty \text{ y } x = +\infty \rangle$ 
  for s  $\in$  S:
    for endpoint p de s:
      if p  $\notin$  Q: Q.insert(p); L(p) = U(p) = C(p) =  $\emptyset$ 
      if p es lower endpoint de s: L(p).append(s)
      if p es upper endpoint de s: U(p).append(s)
  while Q  $\neq \emptyset$ :
    p  $\leftarrow$  Q.nextEvent()
    Q.deleteEvent(p)
    handleEvent(p)

```

2.5 handleEvent — El corazón del algoritmo

```

handleEvent(p):
  1. if |L(p)  $\cup$  U(p)  $\cup$  C(p)| > 1:
      reportar intersección en p, reportar todos los segmentos involucrados
  2. Eliminar L(p)  $\cup$  C(p) de T // segmentos que terminan o
cruzan
  3. Insertar U(p)  $\cup$  C(p) en T en su nuevo orden // segmentos que comienzan o
cruzan
      (el orden se evalúa "ligeramente debajo de  $\varnothing$ ", lo que efectivamente
intercambia
      los segmentos que se cruzaban)
  4. if U(p)  $\cup$  C(p) =  $\emptyset$ : // p es solo un lower endpoint, todos los segmentos se
fueron
      b_left / b_right = vecino izq/der de p en T
      findNewEvent(b_left, b_right, p)
  else:
      s_left = segmento más a la izquierda en U(p)  $\cup$  C(p)
      s_right = segmento más a la derecha en U(p)  $\cup$  C(p)
      b_left = vecino izquierdo de s_left en T
      b_right = vecino derecho de s_right en T
      findNewEvent(b_left, s_left, p)
      findNewEvent(s_right, b_right, p)

```

2.6 findNewEvent

```

findNewEvent(s, s', p):
    if  $s \cap s' = \emptyset$ : return
     $x \leftarrow s \cap s'$ 
    if x está debajo de  $\ell$  o a la derecha de p:
        if  $x \notin Q$ :  $Q.add(x)$ 
        if  $x \in \text{interior-relativo}(s)$ :  $C(x) \leftarrow C(x) \cup \{s\}$ 
        if  $x \in \text{interior-relativo}(s')$ :  $C(x) \leftarrow C(x) \cup \{s'\}$ 

```

¿Por qué "debajo de ℓ o a la derecha de p"? Para no reportar intersecciones que ya procesamos.

2.7 Intuición de por qué funciona

- Cuando un segmento empieza (upper endpoint), se inserta en T y se chequea contra sus nuevos vecinos.
- Cuando un segmento termina (lower endpoint), se elimina de T, y sus ex-vecinos ahora son adyacentes $\rightarrow$ se chequean entre sí.
- Cuando segmentos se cruzan, se intercambian en T $\rightarrow$ se chequean contra sus nuevos vecinos.

La invariante es: **toda intersección futura se detecta porque los segmentos involucrados se volverán adyacentes en T antes o en el momento de cruzarse.**

2.8 Complejidad

- **Tiempo:** $O((n + I) \log n)$ donde I = número de intersecciones.
- **Espacio:** $O(n + I)$.
- Cada operación en Q y T cuesta $O(\log n)$. El número total de eventos es $O(n + I)$.

2.9 Cómo resolver ejercicios de Line Segment Intersection

Tipo: "Ejecute el sweep line sobre estos segmentos"

1. Listar todos los endpoints, ordenarlos de arriba a abajo (y de izquierda a derecha).
2. Para cada evento, dibujar el estado de T y Q.
3. Mostrar cuándo se agregan/eliminan segmentos de T.
4. Mostrar cuándo se llama a findNewEvent y qué intersecciones se descubren.
5. Reportar las intersecciones encontradas.

Tip: Dibujar la sweep line en cada paso y el orden de los segmentos en T.

3.1 Motivación: Art Gallery Problem

Problema: ¿Cuántas cámaras se necesitan para vigilar una galería (polígono simple) con n vértices?

Visibilidad: Dos puntos en un polígono simple se ven mutuamente si el segmento que los conecta está contenido en el polígono.

Resultado clave: encontrar el mínimo de cámaras es NP-hard. Pero podemos acotar:

Teorema (Art Gallery): $\lfloor n/3 \rfloor$ cámaras siempre son suficientes para cualquier polígono simple con n vértices.

Prueba (idea):

1. Toda triangulación del polígono tiene exactamente $n - 2$ triángulos.
2. El grafo dual de la triangulación es un árbol.
3. Los vértices de la triangulación se pueden 3-colorear (por inducción sobre el árbol dual).
4. La clase de color más pequeña tiene a lo sumo $\lfloor n/3 \rfloor$ vértices $\rightarrow$ colocar cámaras ahí.

3.2 Triangulación: Definiciones

Triangulación de un polígono simple P: descomposición de P en triángulos disjuntos mediante un conjunto maximal de diagonales no cruzadas.

Diagonal: segmento abierto que conecta dos vértices de P y está completamente en el interior de P.

Teorema: Todo polígono simple admite una triangulación, y cualquier triangulación de un polígono con n vértices tiene exactamente $n - 2$ triángulos y $n - 3$ diagonales.

Prueba: Inducción sobre n . Base $n = 3$ trivial. Para $n > 3$, considerar el vértice más a la izquierda v y sus vecinos u, w . O bien uw es diagonal (caso 1), o hay vértices dentro de $\triangle uvw$ y se elige el más cercano a v (caso 2). En ambos casos se divide el polígono y se aplica la hipótesis inductiva.

3.3 Estrategia en $O(n \log n)$: Monotone Decomposition + Triangulation

Paso 1: Particionar P en subpolígonos y-monótonos $\rightarrow O(n \log n)$ **Paso 2:** Triangular cada subpolígono monótono $\rightarrow O(n)$ cada uno, $O(n)$ total

¿Qué es y-monótono?

Un polígono simple es **y-monótono** si toda línea horizontal lo intersecta en un conjunto conexo (o vacío). Equivalentemente, tiene un vértice superior, un vértice inferior, y dos cadenas monótonas (izquierda y derecha) que van del tope al fondo.

Propiedad: Un polígono simple sin vértices **split** ni **merge** es y-monótono.

3.4 Clasificación de vértices

Para clasificar un vértice v con vecinos u (anterior) y w (siguiente) en la frontera:

Tipo	Condición	Descripción visual
Start	u, w están debajo de v ; ángulo interior $< \pi$	Punta hacia arriba, polígono se abre
End	u, w están arriba de v ; ángulo interior $< \pi$	Punta hacia abajo, polígono se cierra
Split	u, w están debajo de v ; ángulo interior $> \pi$	Punta hacia arriba, polígono se divide
Merge	u, w están arriba de v ; ángulo interior $> \pi$	Punta hacia abajo, polígono se une
Regular	uno arriba, uno abajo	Punto intermedio en una cadena

Regla simplificada para el ángulo:

- Si ambos vecinos están debajo y v hace turn left $\rightarrow$ start.
- Si ambos vecinos están debajo y v hace turn right $\rightarrow$ split.
- Si ambos vecinos están arriba y v hace turn left $\rightarrow$ end.
- Si ambos vecinos están arriba y v hace turn right $\rightarrow$ merge.

3.5 Algoritmo: Partición en piezas monótonas (MakeMonotone)

Idea: usar sweep line de arriba a abajo. Para eliminar vértices split y merge, agregar diagonales.

Estructuras:

- **T (Status):** aristas del polígono que tienen el polígono a su derecha, cortadas por ℓ , con su **helper** (el último vértice visitado en ese "componente" que podría conectarse con una diagonal).
- **Q (Eventos):** los n vértices, ordenados de arriba a abajo.

Helper de una arista e : el vértice más bajo sobre la sweep line tal que el segmento horizontal que lo conecta con e está dentro del polígono. Se actualiza durante el sweep.

Manejo de cada tipo de vértice:

Start vertex v :

- Insertar la arista incidente counterclockwise (la que va "hacia abajo") en T con helper = v .

End vertex v :

- Sea e la arista clockwise incidente a v .
- Si $\text{helper}(e)$ es un merge vertex $\rightarrow$ agregar diagonal $(v, \text{helper}(e))$.
- Eliminar e de T .

Split vertex v :

- Encontrar la arista e directamente a la izquierda de v en T .
- Agregar diagonal $(v, \text{helper}(e))$.
- Actualizar $\text{helper}(e) \leftarrow v$.
- Insertar la arista counterclockwise incidente a v en T con $\text{helper} = v$.

Merge vertex v :

- Sea e la arista clockwise incidente a v .
- Si $\text{helper}(e)$ es un merge vertex $\rightarrow$ agregar diagonal $(v, \text{helper}(e))$.
- Eliminar e de T .
- Encontrar la arista e' directamente a la izquierda de v en T .
- Si $\text{helper}(e')$ es un merge vertex $\rightarrow$ agregar diagonal $(v, \text{helper}(e'))$.
- Actualizar $\text{helper}(e') \leftarrow v$.

Regular vertex v :

- Si el polígono está a la derecha de las dos aristas incidentes (la cadena izquierda):
 - Sea e la arista superior. Si $\text{helper}(e)$ es merge $\rightarrow$ diagonal.
 - Reemplazar e por la arista inferior en T , con $\text{helper} = v$.
- Si el polígono está a la izquierda (la cadena derecha):
 - Encontrar e directamente a la izquierda de v . Si $\text{helper}(e)$ es merge $\rightarrow$ diagonal.
 - Actualizar $\text{helper}(e) \leftarrow v$.

Complejidad: $O(n \log n)$ (sort + n operaciones en BST balanceado).

Nota importante: después de este sweep de arriba a abajo, los vértices split se eliminaron. Pero aún pueden quedar vértices merge (que en un sweep de abajo arriba serían split). Por eso se hace un **segundo sweep** de abajo a arriba para manejar los merge restantes, o se integra el manejo en el mismo sweep.

3.6 Triangulación de polígono monótono: $O(n)$

Una vez que tenemos polígonos y-monótonos, triangularlos es lineal.

```
TriangulateMonotonePolygon(P):
```

```
1. Merge left and right chains  $\rightarrow u_1, \dots, u_n$  con  $y_1 \geq \dots \geq y_n$   
2. Stack S; S.push( $u_1$ ); S.push( $u_2$ )  
3. for j  $\leftarrow$  3 to n-1:  
    if  $u_j$  y S.top() están en cadenas diferentes:  
        while not S.empty():  
            v  $\leftarrow$  S.pop()  
            if not S.empty(): dibujar diagonal ( $u_j$ , v)  
            S.push( $u_{j-1}$ ); S.push( $u_j$ )  
    else: // misma cadena  
        v  $\leftarrow$  S.pop()  
        while not S.empty() AND  $u_j$  ve S.top():  
            v  $\leftarrow$  S.pop(); dibujar diagonal ( $u_j$ , v)  
        S.push(v); S.push( $u_j$ )  
4. Dibujar diagonales desde  $u_n$  a todos los vértices en S excepto primero y último.
```

Intuición: Se procesan los vértices de arriba a abajo. El stack mantiene los vértices "pendientes" (reflex chain). Cuando llega un vértice de la cadena opuesta, se puede triangular todo lo acumulado. Cuando llega uno de la misma cadena, se triangulan los que sean visibles (los que no generan ángulo reflejo).

" u_j ve S.top()": significa que la diagonal entre u_j y S.top() está dentro del polígono. Esto ocurre cuando el triángulo formado tiene la orientación correcta (no se sale del polígono).

3.7 Resultado final

Teorema: Un polígono simple con n vértices se puede triangular en $O(n \log n)$.

- Partición en piezas monótonas: $O(n \log n)$
- Triangulación de cada pieza: $O(n)$ total

3.8 Cómo resolver ejercicios de Triangulación

Tipo: "Clasifique los vértices de este polígono"

- Para cada vértice, ver sus vecinos. ¿Están ambos arriba, ambos abajo, o uno y uno?
- Verificar si el ángulo interior es $< \pi$ (convexo) o $> \pi$ (reflex).

Tipo: "Ejecute MakeMonotone"

1. Clasificar todos los vértices.
2. Procesar de arriba a abajo.
3. Para cada vértice, aplicar las reglas correspondientes.
4. Mostrar T y los helpers en cada paso.
5. Indicar qué diagonales se agregan.

Tipo: "Triangule este polígono monótono"

1. Identificar left chain y right chain.
 2. Merge ambas cadenas en orden descendente de y.
 3. Ejecutar el algoritmo con el stack paso a paso.
-

Unidad 4: Linear Programming (2D)

4.1 Formulación general

Problema LP general: Minimizar (o maximizar) $c_1x_1 + c_2x_2 + \dots + c_kx_k$ sujeto a restricciones lineales: $a_{11}x_1 + \dots + a_{1k}x_k \leq b_1, \dots, a_{n1}x_1 + \dots + a_{nk}x_k \leq b_n$

En geometría computacional nos interesa $d = 2$: dos variables, n restricciones (semitplanos).

Vocabulario:

- **Feasible region:** la intersección de todos los semiplanos (región factible). Es convexa.
- **Infeasible:** la región factible es vacía.
- **Unbounded:** existen soluciones con costo arbitrariamente bajo.
- **Objective vector / cost vector:** $(c_1, c_2)^T$, la dirección de optimización.
- **"Profit lines":** líneas de nivel ortogonales al vector objetivo.

4.2 Motivación: Casting (Moldeo)

¿Puede un poliedro ser removido de su molde por una traslación?

Lema: El poliedro P puede removerse del molde en dirección d si y solo si d forma un ángulo $\geq 90^\circ$ con la normal exterior de todas las facetas ordinarias de P .

Esto se reduce a encontrar la intersección de $n-1$ semiplanos $\rightarrow$ **Half-Plane Intersection**.

4.3 Intersección de semiplanos: Divide & Conquer — $O(n \log n)$

```

IntersectHalfPlanes(H):
  if |H| = 1:
    return h (el único semiplano)
  else:
    dividir H en  $H_1$  y  $H_2$  de tamaño  $\lceil n/2 \rceil$  y  $\lfloor n/2 \rfloor$ 
     $C_1 \leftarrow \text{IntersectHalfPlanes}(H_1)$ 
     $C_2 \leftarrow \text{IntersectHalfPlanes}(H_2)$ 
     $C \leftarrow \text{IntersectConvexRegions}(C_1, C_2)$  //  $O(n)$  con sweep line

```

IntersectConvexRegions puede hacerse en $O(n)$ tiempo porque el sweep line status siempre tiene a lo sumo 4 segmentos.

Recurrencia: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ por Master Theorem.

4.4 LP en 2D: Enfoque Incremental

En lugar de computar toda la intersección, solo buscamos un punto óptimo.

```

2DBoundedLP(H):
   $h_1, h_2 \leftarrow$  primeros dos semiplanos;  $v_2 \leftarrow$  primera solución
  for  $i \leftarrow 3$  to  $n$ :
    if  $v_{i-1} \in h_i$ : // el óptimo actual satisface la nueva
restricción
       $v_i \leftarrow v_{i-1}$  //  $O(1)$ 
    else: // el óptimo violó la nueva restricción
       $v_i \leftarrow$  punto en  $\ell_i$  que minimiza y sujeto a  $H_{i-1}$ 
// LP en 1D sobre la línea  $\ell_i \rightarrow O(i)$ 
      if no existe:
        reportar infeasible; quit
  return  $v_n$ 

```

¿Cómo resolver el 1D LP en la línea ℓ_i ? Cada restricción h_j ($j < i$) restringe un intervalo de ℓ_i . Se intersectan todos esos intervalos en $O(i)$, y se busca el óptimo en el intervalo resultante.

Complejidad determinista: $T(n) = \sum_i O(i) = O(n^2)$ en el peor caso.

4.5 LP Randomizado: $O(n)$ esperado

Idea clave: permutar aleatoriamente las restricciones antes de procesarlas.

```

RandomizedLP(H):
   $h_1, h_2 \leftarrow$  primeros dos semiplanos;  $v_2 \leftarrow$  primera solución
   $h_3, \dots, h_n \leftarrow$  en ORDEN ALEATORIO
  for  $i \leftarrow 3$  to  $n$ :
    if  $v_{i-1} \in h_i$ :
       $v_i \leftarrow v_{i-1}$  //  $O(1)$ 
    else:
       $v_i \leftarrow$  1D LP en  $\ell_i$  sujeto a  $H_{i-1}$  //  $O(i)$ 
      if infeasible: quit
  return  $v_n$ 

```

Análisis con Backward Analysis:

Sea $X_i = 1$ si $v_{i-1} \notin h_i$ (el óptimo cambia). El costo esperado es:

$$E[T(n)] = E[\sum (1-X_i) \cdot O(1) + X_i \cdot O(i)] = O(n) + \sum \Pr[X_i=1] \cdot O(i)$$

Backward analysis: ¿Cuál es la probabilidad de que al remover h_i de $H_i = \{h_1, \dots, h_i\}$ cambie el óptimo v_i ? El óptimo v_i está en la intersección de exactamente 2 líneas frontera de H_i (en caso genérico). Al remover una restricción aleatoria de las i , la probabilidad de que sea una de esas 2 es:

$$\Pr[X_i = 1] \leq 2/i$$

Entonces:

$$E[T(n)] \leq O(n) + \sum_{i=3}^n (2/i) \cdot O(i) = O(n) + \sum_{i=3}^n O(1) \cdot 2 = O(n) + O(n) = O(n)$$

Teorema: El LP en 2D acotado se puede resolver en $O(n)$ tiempo esperado.

4.6 Cómo resolver ejercicios de Linear Programming

Tipo: "Ejecute el LP incremental sobre estas restricciones"

1. Dibujar los semiplanos uno a uno.
2. En cada paso, verificar si el óptimo actual satisface la nueva restricción.
3. Si no, resolver el 1D LP en la línea frontera de la nueva restricción.
4. Mostrar el óptimo en cada paso.

Tipo: "Formule X como un LP"

- Identificar las variables (generalmente coordenadas de un punto).
- Identificar las restricciones (generalmente condiciones geométricas que se traducen en desigualdades lineales).
- Identificar la función objetivo (o si solo es feasibility, elegir cualquier objetivo trivial).

Tipo: "¿Cuál es la probabilidad de que el óptimo cambie?"

- Usar backward analysis: $Pr = 2/i$.
-

Unidad 5: Orthogonal Range Searching

5.1 El Problema

Input: conjunto P de n puntos en R^d (nos interesan $d = 1$ y $d = 2$). **Query:** dado un rango ortogonal (intervalo en 1D, rectángulo alineado con los ejes en 2D), reportar todos los puntos de P dentro del rango.

Es un problema de preprocessing: los puntos se conocen de antemano; las queries llegan después. Queremos una estructura de datos que permita queries rápidas.

Métricas: construction time, storage, query time (expresado como $f(n) + O(k)$ donde $k = \text{output size}$).

5.2 Range Queries en 1D

Estructura: Balanced Binary Search Tree (BST) donde las hojas almacenan los puntos ordenados y los nodos internos almacenan valores de split.

Tipos de nodos durante una query $[x, x']$:

- **White:** nunca visitados $\rightarrow$ sin costo.
- **Grey:** visitados, pero no está claro si conducen a output $\rightarrow$ determinan la dependencia en n .
- **Black:** visitados, todo su subárbol es output $\rightarrow$ determinan la dependencia en k .

Algoritmo FindSplitNode

```
FindSplitNode(T, x, x'):
  v ← root(T)
  while v no es hoja AND (x' ≤ x_v OR x > x_v):
    if x' ≤ x_v: v ← lc(v)
    else:      v ← rc(v)
  return v
```

El split node es donde los caminos de búsqueda de x y x' divergen.

Algoritmo 1DRangeQuery

```

1DRangeQuery(T, [x : x']):
    v_split ← FindSplitNode(T, x, x')
    if v_split es hoja:
        chequear si el punto almacenado está en [x, x']
    else:
        // Seguir camino a x, reportar subárboles a la derecha del camino
        v ← lc(v_split)
        while v no es hoja:
            if  $x \leq x_v$ :
                ReportSubtree(rc(v))    // ← subárbol completo = black nodes
                v ← lc(v)
            else:
                v ← rc(v)
        Chequear hoja v.
        // Análogamente, seguir camino a x' y reportar subárboles a la izquierda.

```

Análisis:

- Grey nodes: están en dos caminos de la raíz a hojas $\rightarrow O(\log n)$
- Black nodes: un subárbol con m hojas tiene $m-1$ nodos internos $\rightarrow$ traversal $O(m)$ para m puntos de output
- Query time: $O(\log n + k)$

Teorema (1D):

Métrica	Valor
Construction	$O(n \log n)$
Storage	$O(n)$
Query	$O(\log n + k)$

5.3 Kd-Trees (2D)

Idea: dividir el conjunto de puntos alternando entre coordenada x e y . En cada nivel, se parte por la mediana.

- Niveles pares: split por x (línea vertical por la mediana- x).
- Niveles impares: split por y (línea horizontal por la mediana- y).

```

BuildKDTree(P, depth):
    if |P| = 1:
        return hoja con el punto
    else:
        if depth es par:
             $\ell \leftarrow$  línea vertical por mediana-x de P
             $P_1 \leftarrow$  puntos a la izquierda o sobre  $\ell$ 
             $P_2 \leftarrow$  puntos a la derecha de  $\ell$ 
        else:
             $\ell \leftarrow$  línea horizontal por mediana-y de P
             $P_1 \leftarrow$  puntos debajo o sobre  $\ell$ 
             $P_2 \leftarrow$  puntos arriba de  $\ell$ 
        v_left  $\leftarrow$  BuildKDTree( $P_1$ , depth + 1)
        v_right  $\leftarrow$  BuildKDTree( $P_2$ , depth + 1)
        return nodo v con  $\ell$ , hijos v_left y v_right

```

Construction: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ (si se usa mediana lineal o pre-sort).

Query en Kd-tree

```

SearchKDTree(v, R):
    if v es hoja:
        reportar el punto si está en R
    else:
        if region(lc(v))  $\subseteq$  R:
            ReportSubtree(lc(v))
        else if region(lc(v))  $\cap$  R  $\neq \emptyset$ :
            SearchKDTree(lc(v), R)
        // Análogo para rc(v)

```

Cada nodo v tiene una **región** asociada (el rectángulo del espacio que representa). Se compara contra el query rectangle R .

Análisis de Query Time — Contar regiones intersectadas:

La recurrencia para el número de regiones que intersectan un borde del rectángulo de query es:

$$Q(n) = 2 + 2 \cdot Q(n/4) \text{ (base } Q(1) = O(1))$$

¿Por qué $n/4$? Porque una línea vertical (borde del query) intersecta las regiones de un nivel x-split (cortando la mitad en dos), pero en el siguiente nivel (y-split), solo una de las dos sub-regiones se intersecta. Así, en dos niveles se pasa de n a $n/4$.

Resolviendo: $Q(n) = O(\sqrt{n})$.

Como hay 4 bordes del rectángulo: **query time** = $O(\sqrt{n} + k)$.

Teorema (Kd-tree):

Métrica	Valor
Construction	$O(n \log n)$
Storage	$O(n)$
Query	$O(\sqrt{n} + k)$

5.4 Range Trees (2D)

Idea: usar un BST balanceado en x-coordenada como estructura principal. Pero en cada nodo interno, almacenar un **BST asociado** en y-coordenada con todos los puntos del subárbol.

Estructura multinivel:

- **Árbol principal:** BST balanceado sobre x-coordenada (hojas = puntos).
- **Estructura asociada:** en cada nodo v del árbol principal, un BST sobre y-coordenada con todos los puntos en el subárbol de v .

Storage

Dos formas de analizarlo:

- **Por nivel:** en cada nivel del árbol principal, cada punto aparece en exactamente una estructura asociada. Hay $O(\log n)$ niveles $\rightarrow O(n \log n)$ total.
- **Por punto:** cada punto aparece en las estructuras asociadas de los $O(\log n)$ nodos en su camino de búsqueda.

Storage: $O(n \log n)$

Construction

Versión naive: $T(n) = 2T(n/2) + O(n \log n) \rightarrow O(n \log^2 n)$

Versión optimizada: pre-ordenar P por y-coordenada. Al dividir en P_{left} y P_{right} , mantener el orden-y. Así la estructura asociada se construye en $O(n)$ por nodo.

$T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$

Query

2DRangeQuery con Range Tree:

1. En el árbol principal (x), hacer un range query 1D para $[x_1, x_2]$.
Esto identifica $O(\log n)$ subárboles canónicos cuyos puntos tienen $x \in [x_1, x_2]$.
2. Para cada subárbol canónico, hacer un range query 1D en su estructura asociada (y) para $[y_1, y_2]$.

Análisis:

- Se visitan $O(\log n)$ nodos grey en el árbol principal.
- En cada uno, se hace un query 1D en la estructura asociada: $O(\log n + k_u)$.
- Total: $\sum O(\log n + k_u) = O(\log^2 n + k)$.

Teorema (Range Tree):

Métrica	Valor
Construction	$O(n \log n)$
Storage	$O(n \log n)$
Query	$O(\log^2 n + k)$

5.5 Comparación Kd-tree vs Range Tree

	Kd-tree	Range Tree
Construction	$O(n \log n)$	$O(n \log n)$
Storage	$O(n)$	$O(n \log n)$
Query	$O(\sqrt{n} + k)$	$O(\log^2 n + k)$

Kd-tree: mejor storage, peor query. **Range tree:** mejor query, peor storage.

Para n grande, $\log^2 n \ll \sqrt{n}$, así que Range Tree es mejor para queries si el espacio no es problema.

5.6 Cómo resolver ejercicios de Range Searching

Tipo: "Construya un Kd-tree para estos puntos"

1. Nivel 0 (raíz): encontrar mediana-x, dividir.
2. Nivel 1: encontrar mediana-y en cada mitad, dividir.
3. Continuar alternando hasta hojas unitarias.
4. Dibujar el árbol y las particiones del espacio.

Tipo: "Ejecute una query $[x_1, x_2] \times [y_1, y_2]$ en el Kd-tree"

1. Desde la raíz, para cada nodo, verificar si su región está contenida en R, si intersecta R, o si es disjunta.
2. Si contenida $\rightarrow$ ReportSubtree.
3. Si intersecta $\rightarrow$ recursar.
4. Si disjunta $\rightarrow$ podar.
5. En las hojas, verificar si el punto está en R.

Tipo: "Construya un Range Tree"

1. Ordenar puntos por x $\rightarrow$ árbol principal.
2. En cada nodo, listar los puntos de su subárbol y construir el BST asociado por y.
3. Mostrar las estructuras asociadas de al menos algunos nodos internos.

Tipo: "Ejecute query en Range Tree"

1. Buscar x_1 y x_2 en el árbol principal $\rightarrow$ encontrar split node.
2. Seguir caminos y marcar los subárboles canónicos (como en 1D range query).
3. En cada subárbol canónico, hacer 1D query en la estructura asociada para $[y_1, y_2]$.

Resumen de Complejidades para el Certamen

Problema	Algoritmo	Tiempo	Espacio
Convex Hull	Graham Scan	$O(n \log n)$	$O(n)$
Convex Hull	Jarvis March	$O(nh)$	$O(n)$
Line Seg. Intersection	Sweep Line	$O((n+I) \log n)$	$O(n+I)$
Polygon Triangulation	Monotone Decomp. + Triangulate	$O(n \log n)$	$O(n)$
2D LP	Randomized Incremental	$O(n)$ esperado	$O(n)$
Half-Plane Intersection	Divide & Conquer	$O(n \log n)$	$O(n)$
1D Range Query	BST	build $O(n \log n)$, query $O(\log n + k)$	$O(n)$
2D Range Query	Kd-tree	build $O(n \log n)$, query $O(\sqrt{n} + k)$	$O(n)$
2D Range Query	Range Tree	build $O(n \log n)$, query $O(\log^2 n + k)$	$O(n \log n)$

Checklist Pre-Certamen

- ☐ ¿Puedo ejecutar Graham Scan paso a paso sobre un conjunto de puntos?
- ☐ ¿Entiendo la primitiva de orientación (left/right turn) y la puedo calcular a mano?
- ☐ ¿Puedo describir los conjuntos $L(p)$, $U(p)$, $C(p)$ y cómo funciona handleEvent?
- ☐ ¿Puedo clasificar los vértices de un polígono en start/end/split/merge/regular?
- ☐ ¿Entiendo el rol del helper en MakeMonotone y cuándo se agregan diagonales?
- ☐ ¿Puedo ejecutar el algoritmo de triangulación de un polígono monótono con el stack?
- ☐ ¿Puedo formular un problema geométrico como LP en 2D?
- ☐ ¿Entiendo el backward analysis y por qué $\Pr[X_i=1] \leq 2/i$?
- ☐ ¿Puedo construir un Kd-tree y ejecutar una range query?
- ☐ ¿Puedo explicar la diferencia de storage y query time entre Kd-tree y Range Tree?
- ☐ ¿Puedo derivar las recurrencias de complejidad de cada estructura?